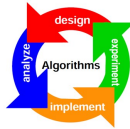


# Estrategias algorítmicas

José A. Rodríguez Melquiades

3 de mayo de 2026

# Metaheurísticas



- 1 **Metaheurísticas**
  - **Tipos de metaheurísticas**
    - Simulated Annealing
    - Variable Neighborhood Search
    - Iterated Local Search

# Metaheurísticas

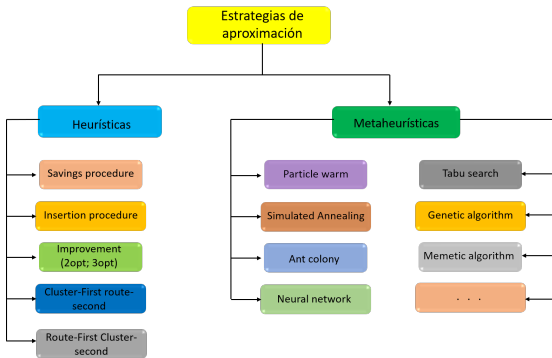


Figura 1: Estrategias aproximadas para solucionar problemas difíciles

# Tipos de metaheurísticas

- **Recocido simulado (Simulated Annealing, SA):** Se basa en el proceso físico de recocido de los metales, por lo que explora el espacio de soluciones aceptando movimientos que empeoran temporalmente la solución actual para evitar quedarse atrapado en óptimos locales.



# Tipos de metaheurísticas

- **Recocido simulado (Simulated Annealing, SA):** Se basa en el proceso físico de recocido de los metales, por lo que explora el espacio de soluciones aceptando movimientos que empeoran temporalmente la solución actual para evitar quedarse atrapado en óptimos locales.
- **Algoritmos Genéticos (Genetic Algorithm, GA):** Basado en la evolución natural. Para hallar una solución de un problema usan operadores como selección, cruzamiento y mutación para generar y mejorar soluciones de generación en generación.

# Tipos de metaheurísticas

- **Recocido simulado (Simulated Annealing, SA):** Se basa en el proceso físico de recocido de los metales, por lo que explora el espacio de soluciones aceptando movimientos que empeoran temporalmente la solución actual para evitar quedarse atrapado en óptimos locales.
- **Algoritmos Genéticos (Genetic Algorithm, GA):** Basado en la evolución natural. Para hallar una solución de un problema usan operadores como selección, cruzamiento y mutación para generar y mejorar soluciones de generación en generación.
- **Búsqueda Tabú:** Evita ciclos en la búsqueda al mantener una lista de movimientos "tabú" que no se pueden revertir temporalmente.

# Tipos de metaheurísticas

- **Recocido simulado (Simulated Annealing, SA):** Se basa en el proceso físico de recocido de los metales, por lo que explora el espacio de soluciones aceptando movimientos que empeoran temporalmente la solución actual para evitar quedarse atrapado en óptimos locales.
- **Algoritmos Genéticos (Genetic Algorithm, GA):** Basado en la evolución natural. Para hallar una solución de un problema usan operadores como selección, cruzamiento y mutación para generar y mejorar soluciones de generación en generación.
- **Búsqueda Tabú:** Evita ciclos en la búsqueda al mantener una lista de movimientos "tabú" que no se pueden revertir temporalmente.
- **Optimización por Enjambre de Partículas (Particle Swarm Optimization, PSO):** Modela el comportamiento de las aves en busca de comida, donde cada "partícula" representa una solución potencial y se mueve por el espacio de búsqueda en función de su mejor posición local y global.

## Optimización por Enjambre de Partículas (Particle Swarm Optimization, PSO)

Modela el comportamiento de las aves en busca de comida, donde cada "partícula" representa una solución potencial y se mueve por el espacio de búsqueda en función de su mejor posición local y global.

## IDEA PRINCIPAL

Un enjambre de aves explora el espacio en busca de comida (la mejor solución)



- Aves explorando (soluciones)
- Mejor posición individual ( $pBest_i$ )
- ★ Mejor posición global ( $gBest$ )

## ALGORITMO PSO

- 

### Inicialización

Generar un enjambre de  $N$  partículas con posiciones y velocidades aleatorias.
- 

### Evaluación

Calcular la calidad (fitness) de cada partícula. Actualizar  $pBest_i$  y  $gBest$ .
- 

### Actualización de velocidad

Cada partícula  $i$  ajusta su velocidad considerando su  $pBest_i$  y el  $gBest$ .

$$v_i^{t+1} = wv_i^t + c_1r_1(pBest_i - x_i^t) + c_2r_2(gBest - x_i^t)$$

→ Inercia (mantener dirección)

→ Cognición (mejora personal)

→ Social (mejora colectiva)
- 

### Actualización de posición

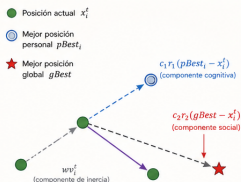
Cada partícula se mueve a una nueva posición.

$$x_i^{t+1} = x_i^t + v_i^{t+1}$$
- 

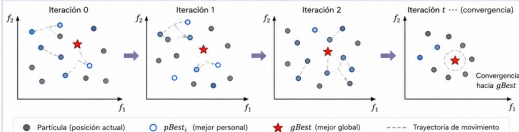
### Criterio de parada

Repetir hasta alcanzar el número máximo de iteraciones o la convergencia.

## COMPORTAMIENTO DE UNA PARTÍCULA



## PROCESO DE BÚSQUEDA



### CARACTERÍSTICAS CLAVE

- ✓ Búsqueda basada en cooperación y aprendizaje social.
- ✓ Equilibra exploración (diversidad) y explotación (intensificación).
- ✓ Pocos parámetros y fácil de implementar.
- ✓ Eficiente para problemas continuos y combinatorios.
- ✓ Desempeño sensible a los parámetros:  $w$ ,  $c_1$ ,  $c_2$ .

## PARÁMETROS IMPORTANTES

|            |                                                        |
|------------|--------------------------------------------------------|
| $N$        | Número de partículas (tamaño del enjambre)             |
| $w$        | Peso de inercia (controla exploración vs. explotación) |
| $c_1$      | Coefficiente cognitivo (influencia personal)           |
| $c_2$      | Coefficiente social (influencia del grupo)             |
| $r_1, r_2$ | Números aleatorios en $[0,1]$                          |
| $T$        | Número máximo de iteraciones                           |



**OBJETIVO:** Encontrar la mejor solución (óptimo global) guiando al enjambre hacia las regiones más prometedoras del espacio de búsqueda.

- **Búsqueda de Vecindario Variable (Variable Neighborhood Search, VNS):**  
Itera entre diferentes estructuras de vecindario para explorar y explotar el espacio de búsqueda.

- **Búsqueda de Vecindario Variable (Variable Neighborhood Search, VNS):** Itera entre diferentes estructuras de vecindario para explorar y explotar el espacio de búsqueda.
- **Búsqueda Local Iterada (Iterated Local Search, ILS):** Combina una búsqueda local con una perturbación aleatoria de la solución para escapar de óptimos locales.

- **Búsqueda de Vecindario Variable (Variable Neighborhood Search, VNS):** Itera entre diferentes estructuras de vecindario para explorar y explotar el espacio de búsqueda.
- **Búsqueda Local Iterada (Iterated Local Search, ILS):** Combina una búsqueda local con una perturbación aleatoria de la solución para escapar de óptimos locales.
- **Algoritmos de Colonia de Hormigas (Ant Colony Optimization, ACO):** Se basa en el comportamiento de las hormigas cuando buscan caminos óptimos que los guíen a fuentes de comida, utiliza feromonas y heurísticas para construir y mejorar soluciones.

## Algoritmos de Colonia de Hormigas (Ant Colony Optimization, ACO)

Se basa en el comportamiento de las hormigas cuando buscan caminos óptimos que los guíen a fuentes de comida, utiliza feromonas y heurísticas para construir y mejorar soluciones.

## IDEA PRINCIPAL



- Las hormigas exploran diferentes caminos.
- Depositan feromonas en los caminos que recorren.
- Los caminos con más feromonas atraen a más hormigas.
- Con el tiempo, se refuerzan los mejores caminos (óptimos).

## ESQUEMA DEL ALGORITMO ACO

-  **Inicialización**  
Inicializar feromonas en todas las aristas ( $\tau_{ij}$ ).
-  **Construcción de soluciones**  
Cada hormiga construye una solución paso a paso eligiendo el siguiente nodo probabilísticamente.
-  **Evaluación**  
Evaluar la calidad de las soluciones construidas (longitud, costo, etc.).
-  **Actualización de feromonas**
  - Evaporación: reduce feromonas en todas las aristas.
  - Depósito: aumenta feromonas en las aristas usadas por mejores soluciones.
-  **Criterio de parada**  
Repetir hasta cumplir el criterio (iteraciones, tiempo, convergencia, etc.).

## COMPONENTES CLAVE

Feromonas ( $\tau_{ij}$ )

Representan la experiencia colectiva.  
Valores altos indican caminos prometedores.

Baja feromona  Alta feromona

Heuristica ( $\eta_{ii}$ )

Información problem-específica que indica la  
deseabilidad inmediata (ej.: 1 / distancia).

### Probabilidad de transición

La hormiga  $k$  en el nodo  $i$  elige el siguiente nodo  $j$  con probabilidad:

$$P_{ij}^k = \frac{(\tau_{ij})^\alpha (\eta_{ij})^\beta}{\sum_{l \in N_i^k} (\tau_{il})^\alpha (\eta_{il})^\beta}$$

donde  $\alpha$  y  $\beta$  controlan la importancia relativa.

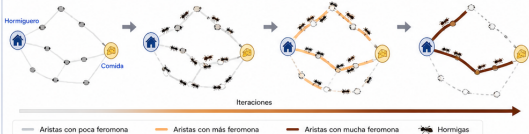
### CÓMO FUNCIONA: EJEMPLO DE BÚSQUEDA DE RUTA ÓPTIMA

- ...una cantidad de feromona.

- amigas exploran diferentes caminos.

- Los caminos más cortos/eficientes  
reciben más feromonas.

- el tiempo, la colonia converge al mejor camino.



## CARACTERÍSTICAS

- ✓ Busca soluciones mediante cooperación indirecta (feromonas).
- ✓ Equilibra exploración (nuevos caminos) y explotación (reforzar buenos caminos).
- ✓ Robusto y flexible para diferentes problemas (TSP, ruteo de vehículos, asignación, scheduling, etc.).
- ✓ El rendimiento depende de parámetros:  $\alpha$ ,  $\beta$ ,  $\rho$  (evaporación) y número de hormigas.

## PARÁMETROS IMPORTANTES

- $m$ : número de hormigas
- $\alpha$ : importancia de la feromona
- $\beta$ : importancia de la heurística
- $\rho$ : tasa de evaporación ( $0 < \rho < 1$ )
- $Q$ : cantidad de feromona depositada



**OBJETIVO:** Encontrar soluciones de alta calidad para problemas de optimización combinando la exploración del espacio de soluciones con el aprendizaje colectivo de la colonia.



- Algoritmos de Recocido Simulado Adaptativo (Adaptive Simulated Annealing, ASA): Es una variante del SA que ajusta parámetros automáticamente durante la búsqueda.

- **Algoritmos de Recocido Simulado Adaptativo (Adaptive Simulated Annealing, ASA):** Es una variante del SA que ajusta parámetros automáticamente durante la búsqueda.
- **Algoritmos Evolutivos Multiobjetivo (Multi-Objective Evolutionary Algorithms, MOEA):** Extienden GA para optimizar múltiples objetivos simultáneamente, generando un conjunto de soluciones no dominadas (Pareto-óptimas).

## Simulated Annealing - SA

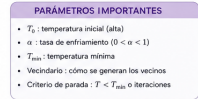
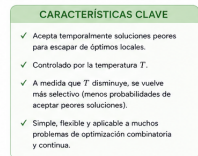
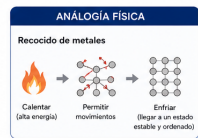
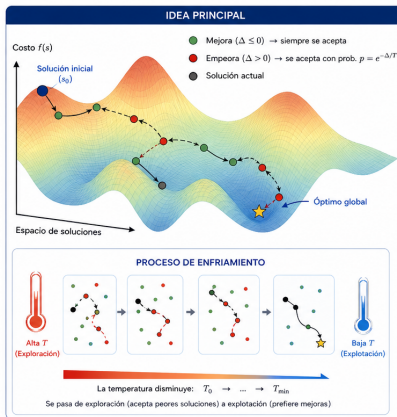
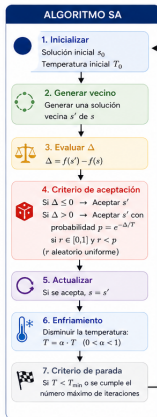
- **SA:** Metaheurística de optimización basado en el proceso físico de recocido de metales, donde el material se calienta y luego se enfría lentamente para minimizar sus imperfecciones y alcanzar un estado de baja energía (óptimo).

## Simulated Annealing - SA

- **SA:** Metaheurística de optimización basado en el proceso físico de recocido de metales, donde el material se calienta y luego se enfría lentamente para minimizar sus imperfecciones y alcanzar un estado de baja energía (óptimo).
- **Idea:** Aceptar soluciones peores de manera controlada para escapar de óptimos locales y eventualmente encontrar una mejor solución global.

# Recocido Simulado (Simulated Annealing, SA)

Se basa en el proceso físico de recocido de los metales, por lo que explora el espacio de soluciones aceptando movimientos que empeoran temporalmente la solución actual para evitar quedarse atrapado en óptimos locales.



**OBJETIVO:** Encontrar una solución de alta calidad explorando el espacio de soluciones de manera inteligente, equilibrando exploración (diversificación) y explotación (intensificación).

- Seudocódigo:

SA( )

- 1: Inicializar solución actual  $S$  ▷ elegir una forma aleatoria o heurística
- 2:  $Costo(S)$  ▷ Evaluar costo de  $S$
- 3:  $T :=$  Valor alto ▷ Inicializar temperatura alta
- 4: **while** No se cumple un criterio de parada **do** ▷ temp. muy baja, núm. de iter o no mejora en varias iter.
- 5:     Generar una solución vecina  $S'$
- 6:      $Costo(S')$  ▷ Evaluar el costo de  $S'$
- 7:      $\Delta E = Costo(S') - Costo(S)$
- 8:     **if**  $\Delta E < 0$  **then**
- 9:         Aceptar  $S'$  como la nueva solución actual
- 10:    **else** Aceptar  $S'$  con probabilidad  $p = e^{-\Delta E/T}$
- 11:    **end if**
- 12:    Disminuir la temperatura  $T$  según esquema de enfriamiento
- 13: **end while**
- 14: Retornar solución

- Ejemplo:

- Ejemplo:
  - ▶ Datos: Clientes: 10

Vehículos: 3



## ● Ejemplo:

► Datos: Clientes: 10

Vehículos: 3

### ► Solución:

Vehículo 1: 0  $\rightarrow$  5  $\rightarrow$  4  $\rightarrow$  6  $\rightarrow$  1  $\rightarrow$  0

Vehículo 2: 0  $\rightarrow$  2  $\rightarrow$  7  $\rightarrow$  10  $\rightarrow$  0

Vehículo 3: 0  $\rightarrow$  3  $\rightarrow$  8  $\rightarrow$  9  $\rightarrow$  0

Distancia total: 409.27

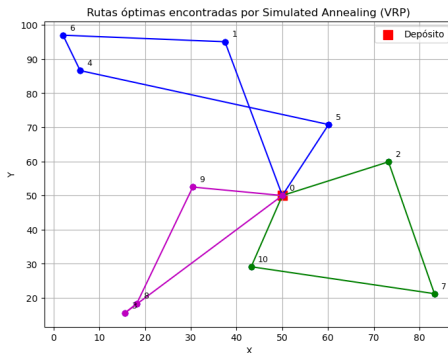


Figura 2: Rutas cubiertas por tres vehículos

## Variable Neighborhood Search - VNS

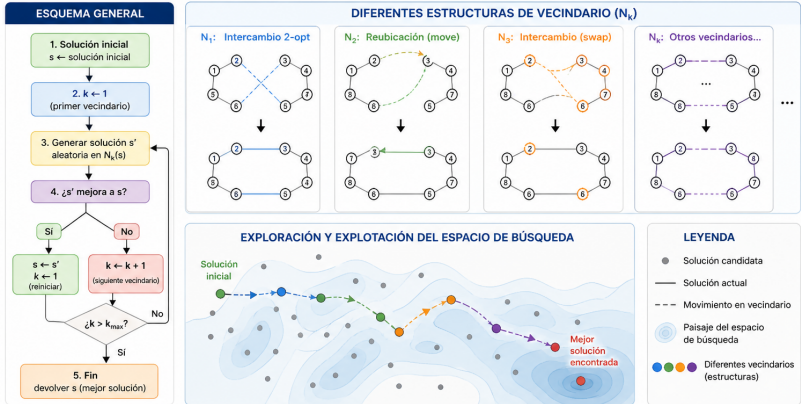
- **Idea:** Explorar sistemáticamente diferentes vecindarios, con el objetivo de hallar mejores soluciones en un espacio de búsqueda. La estrategia clave de VNS es cambiar de un vecindario a otro de manera dinámica, en lugar de fijarse en un solo vecindario, lo que permite escapar de óptimos locales y explorar de manera más efectiva el espacio de soluciones.

## Variable Neighborhood Search - VNS

- **Idea:** Explorar sistemáticamente diferentes vecindarios, con el objetivo de hallar mejores soluciones en un espacio de búsqueda. La estrategia clave de VNS es cambiar de un vecindario a otro de manera dinámica, en lugar de fijarse en un solo vecindario, lo que permite escapar de óptimos locales y explorar de manera más efectiva el espacio de soluciones.
- El algoritmo de VNS se basa bajo el principio de que el espacio de soluciones está formado por un conjunto de vecindarios, y que cada vecindario corresponde a una modificación simple en la solución. En cada iteración, VNS selecciona un vecindario diferente y realiza una búsqueda local en ese vecindario. Si se encuentra una mejor solución, el algoritmo sigue buscando en ese vecindario; si no, pasa a otro vecindario.

# Búsqueda de Vecindario Variable (VNS)

Itera entre diferentes estructuras de vecindario para explorar y explotar el espacio de búsqueda.



**IDEA CLAVE:** VNS cambia sistemáticamente entre vecindarios de diferente tamaño/estructura.

- Vecindarios pequeños → explotación (búsqueda local)
- Vecindarios grandes → exploración (saltos más lejanos)

→ Equilibrio entre exploración y explotación

- Seudocódigo:

VNS( )

- 1: Inicializar solución  $S$
- 2: Establecer el número máximo de iteraciones sin mejora
- 3: Definir vecindarios  $N_1, N_2, \dots, N_k$
- 4: **while** No se cumple un criterio de parada **do**
- 5:      $i = 1$
- 6:     **while**  $i \leq k$  **do** ▷ Exploración de vecindarios
- 7:         Generar solución vecina  $S'$  en el vecindario  $N_i$
- 8:         **if** La solución  $S'$  es mejor que la solución  $S$  **then** ▷ Cambio de vecindario
- 9:             Establecer  $S = S'$
- 10:            Volver a  $i = 1$  ▷ Retornar a explorar el 1er. vecindario
- 11:            Continuar con el siguiente vecindario
- 12:         **end if**
- 13:          $i = i + 1$
- 14:     **end while**
- 15: **end while**
- 16: Retornar solución

- Ejemplo:

- Ejemplo:

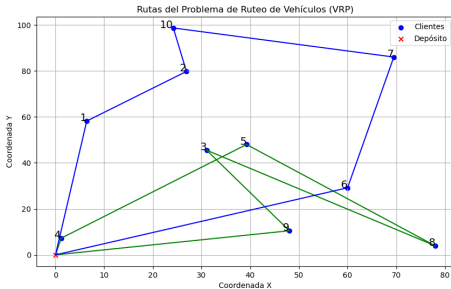
- ▶ Datos: Clientes: 10

Vehículos: 2

Capacidad vehículo: 20

- Ejemplo:

- **Datos:** Clientes: 10                      Vehículos: 2                      Capacidad vehículo: 20
- **Solución:**
- Vehículo 1:  $\chi \rightarrow 4 \rightarrow 5 \rightarrow 8 \rightarrow 3 \rightarrow 9 \rightarrow \chi$
- Vehículo 2:  $\chi \rightarrow 1 \rightarrow 2 \rightarrow 10 \rightarrow 7 \rightarrow 6 \rightarrow \chi$
- Distancia total: 551.35



### Figura 3: Rutas cubiertos por dos vehículos



## Iterated Local Search - ILS

- **Idea:** Aplica la busca local estándar para hallar un óptimo local, luego perturbar ligeramente la solución obtenida para escapar de mínimos locales y seguir buscando mejores soluciones.

## Iterated Local Search - ILS

- **Idea:** Aplica la busca local estándar para hallar un óptimo local, luego perturbar ligeramente la solución obtenida para escapar de mínimos locales y seguir buscando mejores soluciones.
- **Etapas:**

## Iterated Local Search - ILS

- **Idea:** Aplica la busca local estándar para hallar un óptimo local, luego perturbar ligeramente la solución obtenida para escapar de mínimos locales y seguir buscando mejores soluciones.
- **Etapas:**
  - ▶ **Generar solución inicial:** Aleatoria o construida a través de una heurística.

## Iterated Local Search - ILS

- **Idea:** Aplica la busca local estándar para hallar un óptimo local, luego perturbar ligeramente la solución obtenida para escapar de mínimos locales y seguir buscando mejores soluciones.
- **Etapas:**
  - ▶ **Generar solución inicial:** Aleatoria o construida a través de una heurística.
  - ▶ **Busca local:** Mejorar la solución actual mediante un algoritmo de busca local (Hill Climbing o First Improvement).

## Iterated Local Search - ILS

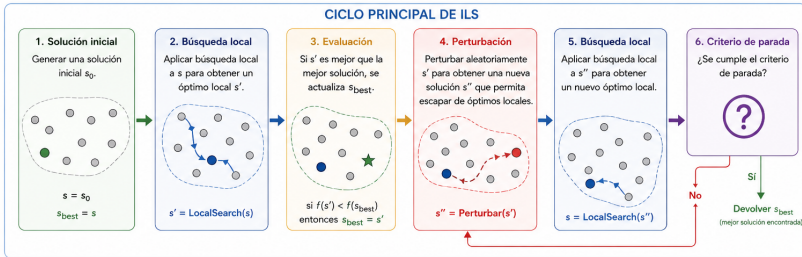
- **Idea:** Aplica la busca local estándar para hallar un óptimo local, luego perturbar ligeramente la solución obtenida para escapar de mínimos locales y seguir buscando mejores soluciones.
- **Etapas:**
  - ▶ **Generar solución inicial:** Aleatoria o construida a través de una heurística.
  - ▶ **Busca local:** Mejorar la solución actual mediante un algoritmo de busca local (Hill Climbing o First Improvement).
  - ▶ **Perturbar:** Modifica la solución actual para escapar de óptimos locales.

## Iterated Local Search - ILS

- **Idea:** Aplica la busca local estándar para hallar un óptimo local, luego perturbar ligeramente la solución obtenida para escapar de mínimos locales y seguir buscando mejores soluciones.
- **Etapas:**
  - ▶ **Generar solución inicial:** Aleatoria o construida a través de una heurística.
  - ▶ **Busca local:** Mejorar la solución actual mediante un algoritmo de busca local (Hill Climbing o First Improvement).
  - ▶ **Perturbar:** Modifica la solución actual para escapar de óptimos locales.
  - ▶ **Aceptar:** Decidir si la nueva solución se acepta como base para continuar.

## Búsqueda Local Iterada (ILS)

Combina una búsqueda local con una perturbación aleatoria de la solución para escapar de óptimos locales.



Explorar el espacio de búsqueda de forma efectiva combinando intensificación (búsqueda local) y diversificación (perturbación) para encontrar soluciones de alta calidad.

## ● Seudocódigo:

ILS( )

- 1:  $S = \text{GenerarSolucionInicial}( )$  ▷ puede ser aleatoria, heurística
- 2:  $S = \text{BuscaLocal}(S)$  ▷ busca una mejor solución en su vecindario
- 3: **while** No se cumple un criterio de parada **do**
- 4:      $S' = \text{Perturbar}(S)$  ▷ aplicar dos intercambios aleatorios, etc.
- 5:      $S' = \text{BuscaLocal}(S')$
- 6:      $S = \text{Aceptar}(S, S')$
- 7: **end while**
- 8: Retornar solución



- Ejemplo:

- Ejemplo:

- ▶ Datos:

Nodos: 11 (con depósito)

Demanda: 10

Cap. vehículos: 50

## ● Ejemplo:

### ► Datos:

Nodos: 11 (con depósito)

Demanda: 10

Cap. vehículos: 50

### ► Solución:

Ruta 1:  $0 \rightarrow 8 \rightarrow 4 \rightarrow 9 \rightarrow 0$

Ruta 2:  $0 \rightarrow 3 \rightarrow 1 \rightarrow 10 \rightarrow 2 \rightarrow 0$

Ruta 3:  $0 \rightarrow 5 \rightarrow 7 \rightarrow 6 \rightarrow 0$

Costo total: 406.59

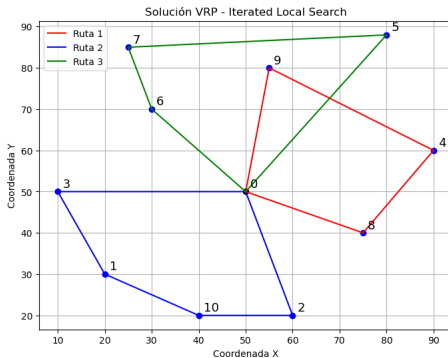


Figura 4: Rutas cubiertas por tres vehículos